MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY

وزارة التعليم
جامعة الإمام
عبد الرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Group #

Group Members:

#	Member Name	Member Academic ID
1		
2		
3		
4		
5		

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Objective

Explore the dataset using PyTorch to understand:

- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

Tasks (with PyTorch)

Task 1: Install PyTorch

Instructions:

- Install PyTorch and load it to your python code and save it as **'Assigment-1 DL'**
- Screenshoot how your load PyTorch to your python source code.

Task 2: Download and Load the Dataset

Instructions:

- PyTorch has some built tools for downloading and loading common datasets. You will be playing around with MNIST in this assignment. It is a dataset of 28x28 grayscale handwritten digits 0-9. There are 50,000 images in the training set and 10,000 in the test set.

- Download the dataset directly from your python code then load it into the the workspace.

Deliverable:

- Screenshot showing the dataset loaded successfully.

Task 3: Inspect the Dataset

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
  1. Images tensor size
  2. Data images vizualization using matplotlib

Task 4: Creating A Simple Network

Instructions:

- Write your Python code to setup the following simple network architecture:

  First you 'll build a very basic neural network with a single hidden layer of neurons. This means one fully connected layer of weights connects the input to the hidden neurons and one fully connected layer connects the hidden neurons to the output. Use the RELU activation function on the hidden neuron values as our nonlinearity.

  These fully connected (or nn.Linear) layers expect a 2D input tensor where that is N x I where N is the number of data points in a **mini batch** and I is the **number of inputs**. However, our data is formatted in (N x C x H x W) right now so we need to tell PyTorch to rearrange it using torch.flatten. C, H and W are refer to Color, Height and Width respectively.

  After being **flattened** our data goes from (32 x 1 x 28 x 28) to being (32 x 784)

Deliverables:

- Screenshot of source code
- Verify your code with above requirement.

Task 5: Training Function

Instructions:

- Now for training the network. Your train function takes as input the net to train and the dataloader for the training data. It also takes some optional parameters to control training.
- For your network you will be using PyTorch's built in nn.CrossEntropyLoss. This will apply a softmax to our network's output, calculate the log-probability assigned to each class, then try to minimize the negative log likelihood of our data (AKA maximize the likelihood)
- For your optimizer you are using stochastic gradient descent with learning rate and momentum

Deliverables:

- Screenshot of your python source code that contains your traing function with above requirements
-

Task 6: Training Network

Instructions:

- You will instantiate a new network and train it on your training data.
- Your training function prints out some debug information about the epoch, batch number, and current loss and accuracy values. It also returns a list of all the losses on your mini-batches so you can plot them all once training has finished.

Deliverables:

- Screenshot of instantiate training source code
- Screenshot of training debug which include epoch, batch number and current loss, accuracy values and graph for overall processes (loss and accuracy for each epoch).

Task 7: Testing Network

Instructions:

- You can test your network on a few of your test images from task 2 and see what happens.

Deliverables:

- Screenshot your network testing source code
- Screenshot the results of the network testing

Task 8: Tuning the parameters and model archirecture

Instructions:

- Tune the parameters with different values of learning rate and momentum and write your best results (lowest loss and highest accuracy).
- Tune the architecture model with different values of number of layer and number of neuron and write your best results (lowest loss and highest accuracy).

Deliverables:

- Screen your source python code with the best results.

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	**0.5**

Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	**0.5**
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	**0.5**
Task 4: Creating A Simple Network	A simple network correctly craeted, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	**2**
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	**1**
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	**1**
Task 8: Tuning the parameters and model archirecture	Tuning parameters and model architecture created, screenshot included, and reults description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture corect but incorrect results description	Not done	**1.5**
Task 9: Save your source code	Source code save succefully	Source code save succefully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	**0.5**
Total					**7.5**